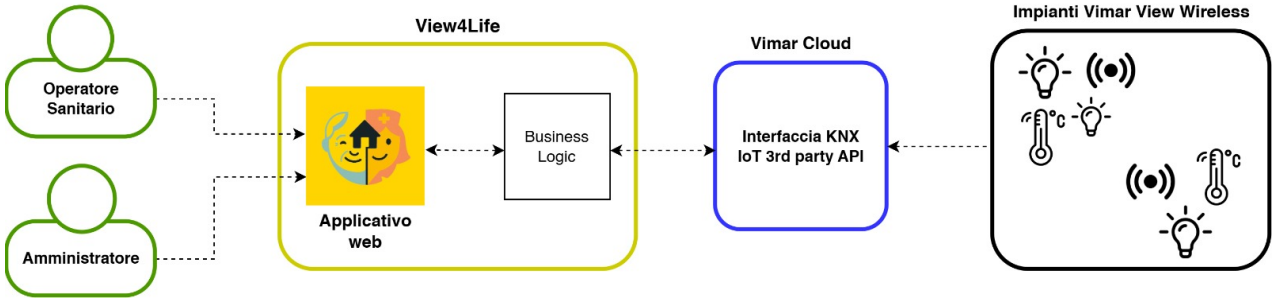
Presentazione architettura

Gruppo 1 - A.A 2025/2026

X/02/2026

Obiettivo capitolato

- Capitolato C9 (Vimar View4Life): Piattaforma per la gestione degli impianti Smart nelle residenze protette;
- Sistema di gestione allarmi e utenti (OSS e Amministrazione) che possono interagire con gli allarmi (gestione e risoluzione)
- Visualizzazione di statistiche sui consumi, con relativi consigli per ridurli, e sugli allarmi.



Architettura di deployment: Monolite

- Non lo so;

Backend: Architettura esagonale

- Problema:
  - La logica di business del *backend* dipende dai dettagli infrastrutturali (*Persistent Logic*);
  - Difficile testare il *backend* senza dover istanziare l'intera infrastruttura.
- Soluzione: **Architettura Esagonale**
  - Il dominio non dipende da componenti esterne;
  - Le **Ports** sono interfacce *TypeScript* che il dominio espone, gli **Adapters** le implementano;
  - Le dipendenze tra i moduli sono risolte tramite il *pattern* **Dependency Injection**.

Frontend: Architettura Model-View-ViewModel

- Problema:
  - Logica di presentazione legata all'interfaccia utente;
  - Ogni cambiamento della vista rischia di impattare la logica di business.
- Soluzione: **Architettura MVVM** che separa le tre responsabilità:
  - **Model**: i dati e la logica di dominio;
  - **View**: il template di Angular;
  - **ViewModel**: un servizio che espone stato e comandi alla View tramite **Observable**.

UML Subscription (Backend)

-

UML Notification (Frontend)

-

Pattern - Dependency Injection (1/2)

- Problema:
  - I moduli sono fortemente accoppiati tra di loro;
  - Il codice risulta difficile da testare e da manutenere.
- Soluzione: pattern **Dependency Injection**
  - Le dipendenze vengono fornite tramite un *injector*;
  - In *TypeScript (Nest JS)* il pattern è supportato nativamente tramite il decoratore *@Injectable()* e il sistema *providers*.

Pattern - Dependency Injection (2/2)

```
1  providers: [
2      { provide: CREATE_ALARM_RULE_USE_CASE, useClass: AlarmRuleService },
3      { provide: CREATE_ALARM_RULE_PORT, useClass: AlarmRulesPersistenceAdapter },
4      { provide: DELETE_ALARM_RULE_USE_CASE, useClass: AlarmRuleService },
5      { provide: DELETE_ALARM_RULE_PORT, useClass: AlarmRulesPersistenceAdapter },
6      { provide: GET_ALL_ALARM_RULES_USE_CASE, useClass: AlarmRuleService },
```

```
1  @Injectable()
2  export class AlarmRuleService
3    implements
4      CreateAlarmRuleUseCase,
5      GetAlarmRuleByIdUseCase,
6      GetAllAlarmRulesUseCase,
7      UpdateAlarmRuleUseCase,
8      DeleteAlarmRuleUseCase,
9      CheckAlarmRuleUseCase
10 {
```

Pattern - Observer (1/3)

- Problema:
  - I componenti *Angular* devono reagire a eventi asincroni (*WebSocket*) e sincronizzare lo stato *UI*.
- Soluzione: pattern **Observer**
  - Implementato tramite *RxJS*: le componenti si iscrivono a *Observable* senza conoscere la sorgente dei dati;
  - Gli *Observable* gestiscono in modo uniforme eventi asincroni e aggiornamenti di stato *UI*;
  - La comunicazione tra componenti avviene tramite *Subject* e *BehaviorSubject* eliminando ogni accoppiamento diretto.

Pattern - Observer (2/3)

UML

Pattern - Observer (2/3)

CODICE